UNIVERSITÄT
LEIPZIG

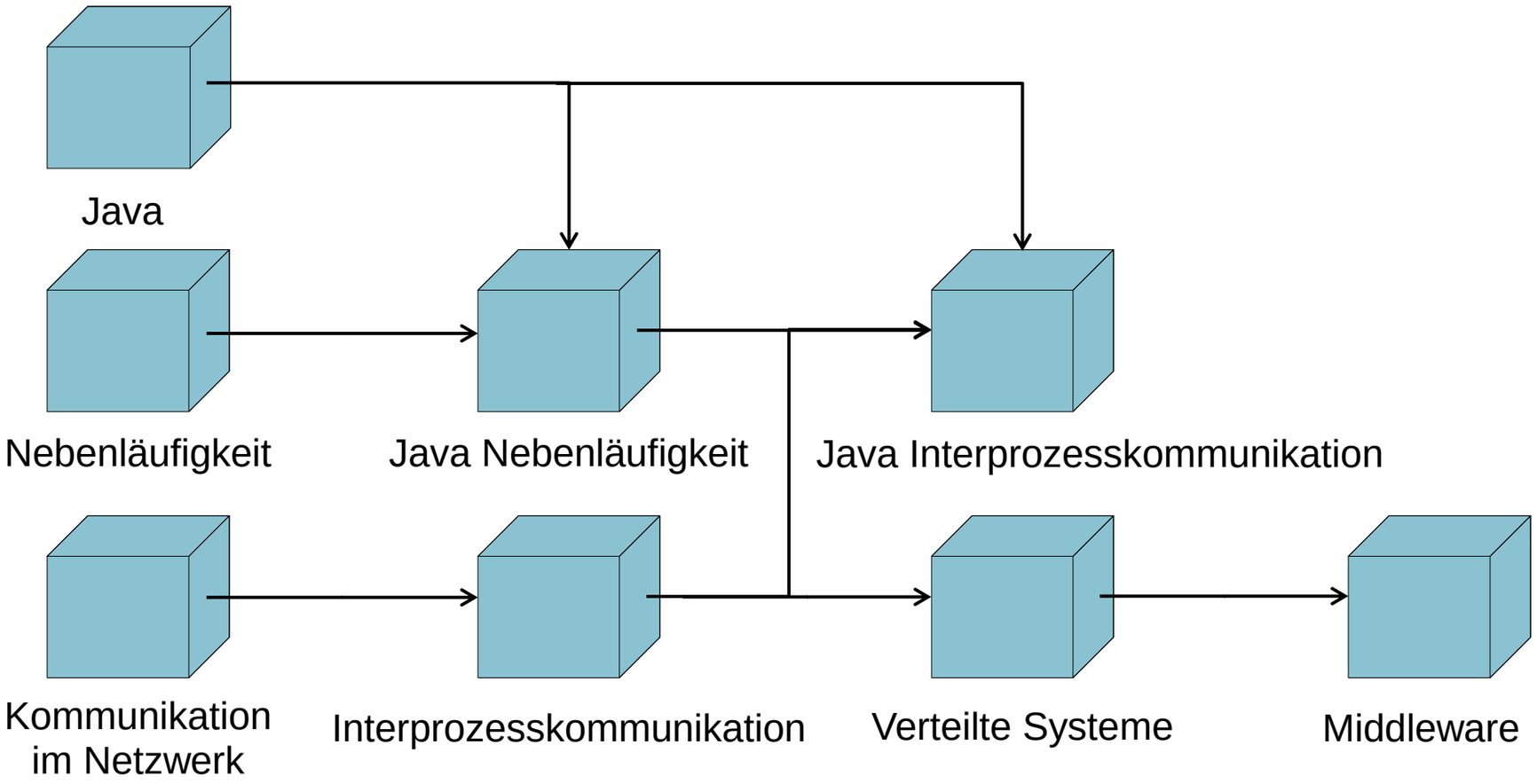
Java Enterprise Edition (JEE)

Entwicklung verteilter Anwendungen

Leipzig, 23.06.2022

Dr. Pascal Kovacs

AGENDA

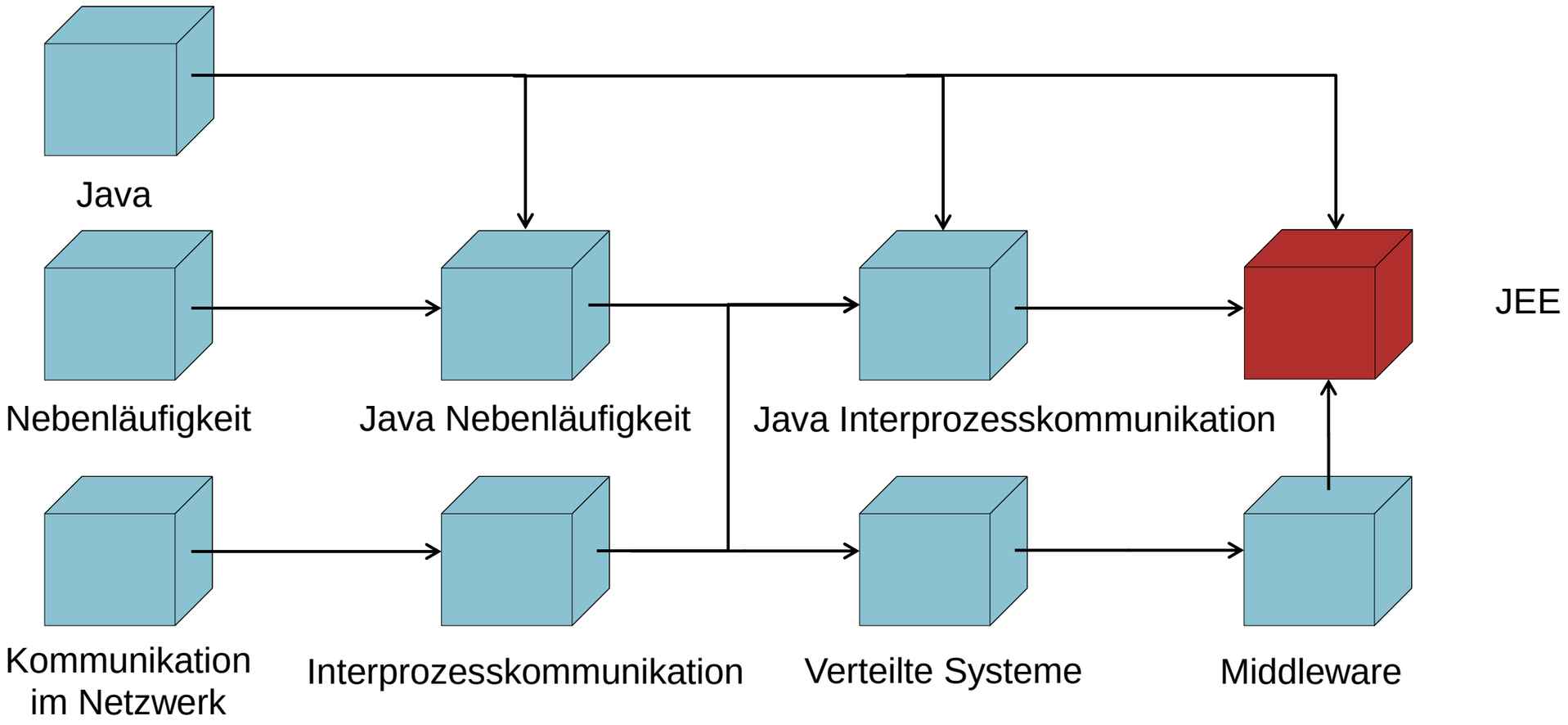

Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
Kommunikation im Netzwerk
Interprozesskommunikation
Verteilte Systeme
Middleware

ZIELE MIDDLEWARE

- Mehrwert kommunikationsorientierter Middleware kennen
- 3 Modelle der kommunikationsorientierten Middleware kennen
- Mehrwert anwendungsorientierter Middleware verstehen
- Dienste von anwendungsorientierter Middleware kennen
- Komponentenparadigma verstehen

BEISPIELFRAGE

AGENDA


Java
Nebenläufigkeit
Java Nebenläufigkeit
Java Interprozesskommunikation
JEE
Kommunikation im Netzwerk
Interprozesskommunikation
Verteilte Systeme
Middleware

ZIELE

- Aufbau JEE Architektur und Application Server kennen
- Komponentenlebenszyklus und dessen Abbildung verstehen
- ~~Enterprise Java Beans mit Vor- und Nachteilen kennen~~
- ~~Funktionsweise der JPA verstehen~~

ÜBERSICHT

- Einführung
- JEE Komponenten-Lebenszyklus

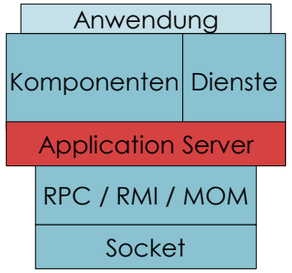
HERAUSFORDERUNGEN UNTERNEHMENSANWENDUNGEN

- Globaler Wettbewerb als zentraler Treiber aller Entwicklungen
- Informationsbedarf und Verarbeitungskapazitäten nehmen stark zu
- Komplexität der Geschäftsprozesse steigt enorm schnell
- Generieren hohe Zahl anspruchsvoller Anforderungen
  - Hochverfügbarkeit, Skalierbarkeit und Sicherheit
  - Speicherung, Verwaltung und Auswertung von Massendaten
  - Mehrere Frontends für Kunden, Beschäftigte und Lieferanten
  - Integration neuer und alter Informationssysteme
  - Lokalisierung von Sprache, Regeln und Gesetzen

JAVA/JAKARTA ENTERPRISE EDITION (JEE)

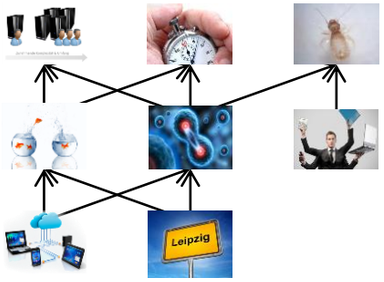
JAKARTA EE
Seit 2019

- JEE als Javas Antwort auf die Herausforderungen von Unternehmen
  - Spezifikation zur Entwicklung von Geschäftsanwendungen
  - Erste Version mit Fokus auf Unternehmensbedürfnisse 1999
  - Technologiestapel verschiedener Hersteller realisiert Spezifikation
  - Kontinuierliche Weiterentwicklung (z.B. Webservices)
  - JEE im Laufe der Zeit umfangreicher, einfacher und portabler
- Vollständige komponentenbasierte Middlewareplattform
  - Nimmt Entwickler zahlreiche technische Aufgaben ab
  - Stellt zusätzliche Dienste über APIs zur Verfügung
- Kommerzielle Implementierung: WebLogic, WebSphere
- Open Source Implementierung: GlassFish, WildFly

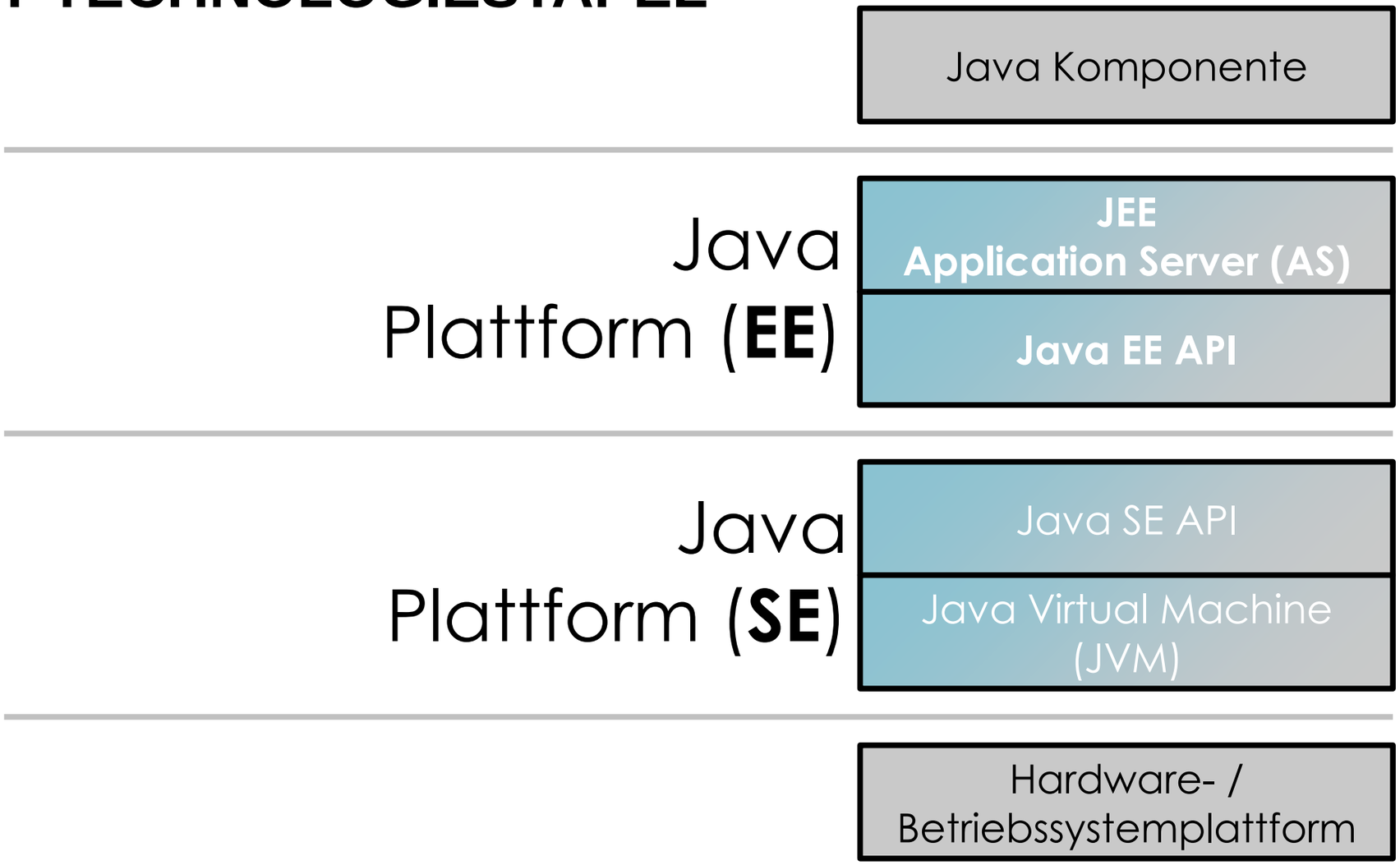


JEE APPLICATION SERVER

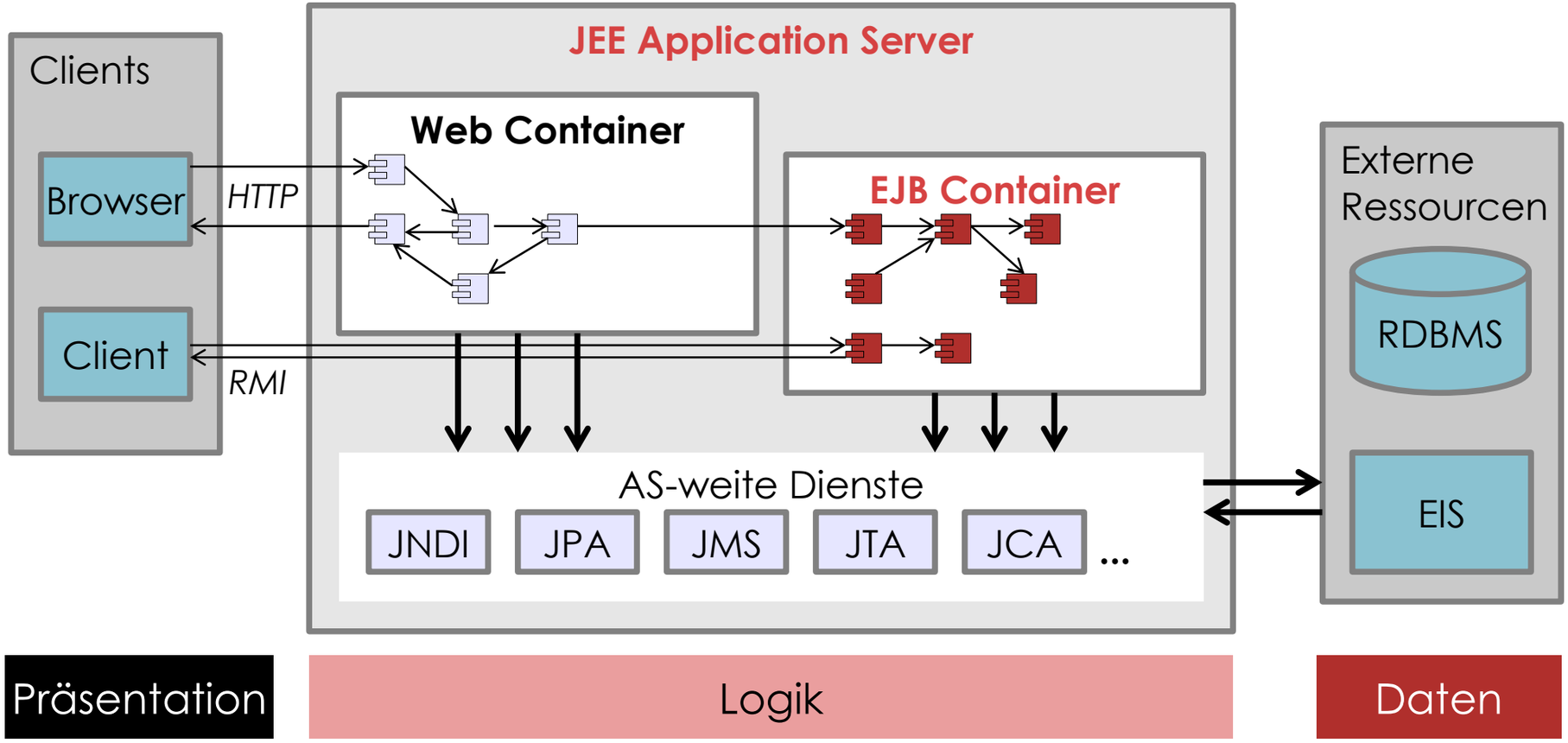
- Realisiert die Java Enterprise Edition (herstellerspezifisch)
- Erzeugt/Verwaltet die Laufzeitumgebung des Komponentenmodells
- Plattform für umfangreiche und komplexe verteilte Anwendungen
  - Komponentenlebenszyklusmanagment
  - Ressourcenverwaltung
  - Load Balancing, Fail over, Caching
- Sammlung von Technologieimplementierung
  - Web Application Technologies (JSF, JSP)
  - Enterprise Application Technologies (EJB, JMS, JPA)
  - Web Services Technologies (REST, Enterprise Web Services)
  - Management and Security Technologies



ÜBERSICHT TECHNOLOGIESTAPEL



ÜBERSICHT ARCHITEKTUR JEE AS



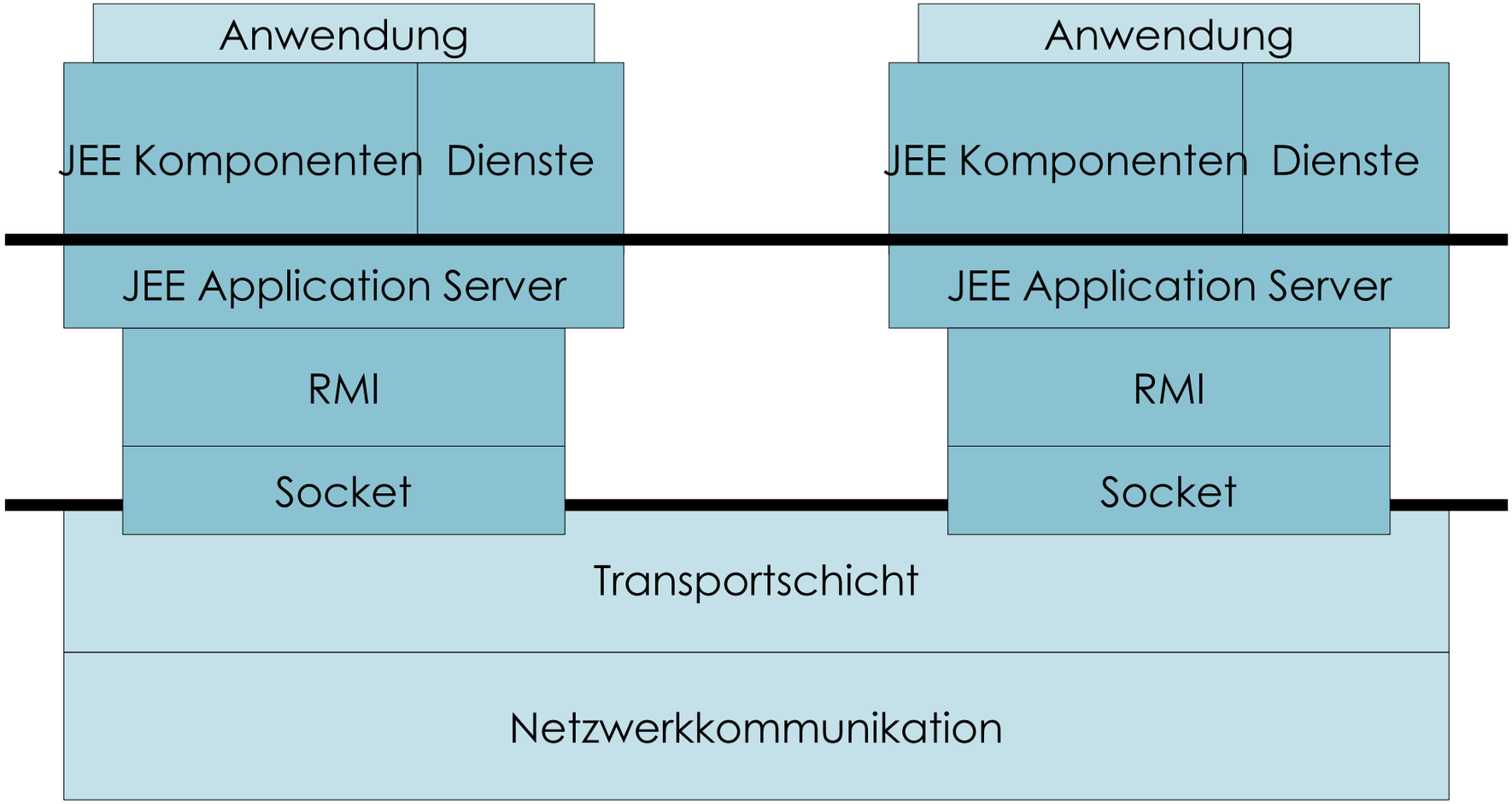
IMPLIZITE DIENSTE VOM CONTAINER AUSGEFÜHRT

- Komponentenlebenszyklus - *component life cycle*
  - Lebenszyklusmanagement der einzelnen Komponenten
- Abhängigkeitsauflösung - *dependency injection*
  - Laden von Ressourcen in eine Komponente - „injizieren“
- Sicherheit - *security*
  - Zugriffskontrolle auf Klassen- und Methodenebene nach Berechtigungen
- Poolverwaltung - *pooling*
  - Vorratsverwalten von häufig nur kurz verwendeten Komponenten
- ...

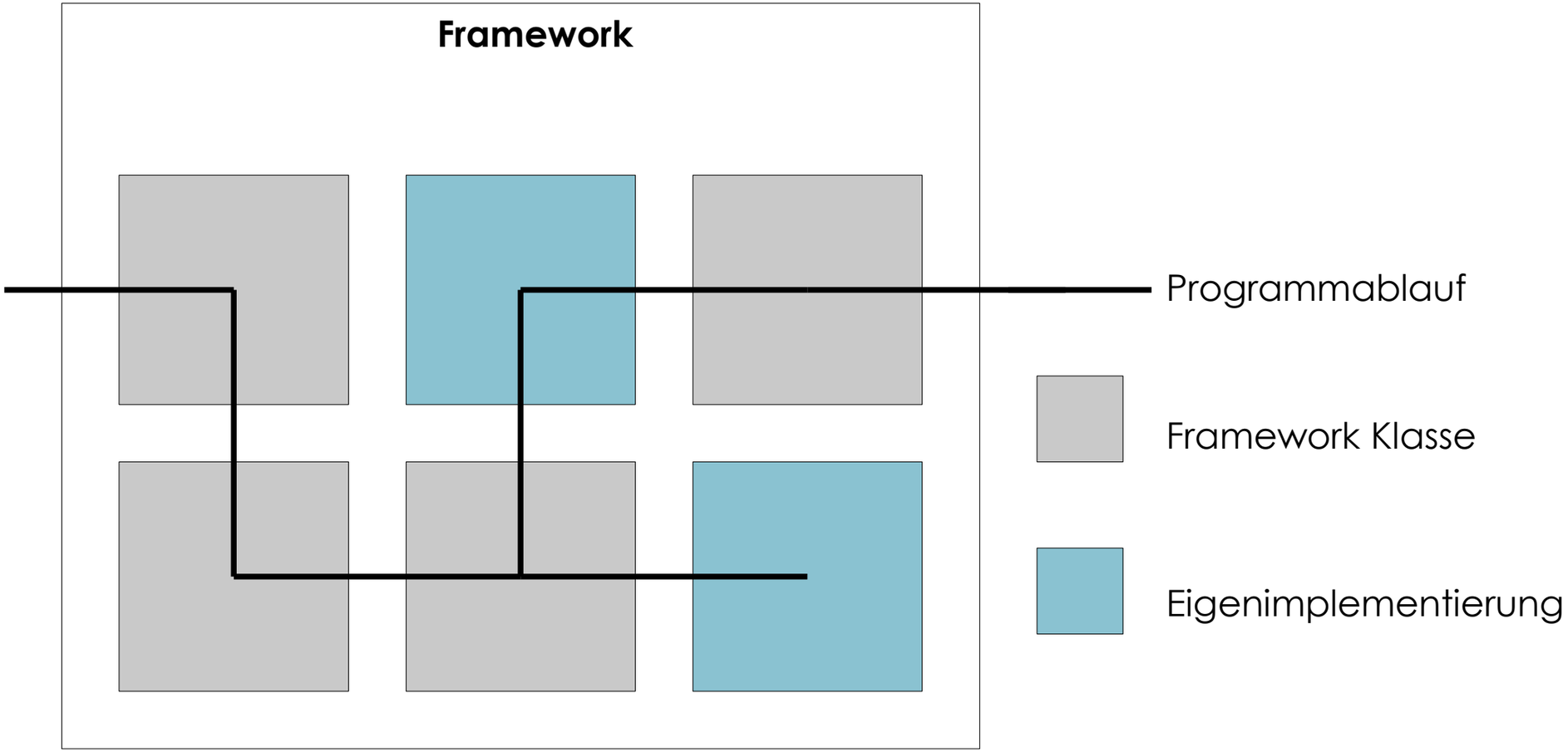
EXPLIZITE DIENSTE VON KOMPONENTE AUFGERUFEN

- JTA (Java Transaction API)
  - Fachliche Transaktionen realisieren
- JPA (Java Persistence API)
  - Persistenzdienst für Objekt-relationales Mapping (ORM)
- JMS (Java Message Service API)
  - Zuverlässige asynchrone verteilte Nachrichtenkommunikation
- JNDI (Java Naming and Directory Interface)
  - Zugriff auf Namens- und Verzeichnisdienst
- …

JEE ALS ANWENDUNGSORIENTIERTE MIDDLEWARE



KOMPONENTEN-FRAMEWORK
INVERSE OF CONTROL



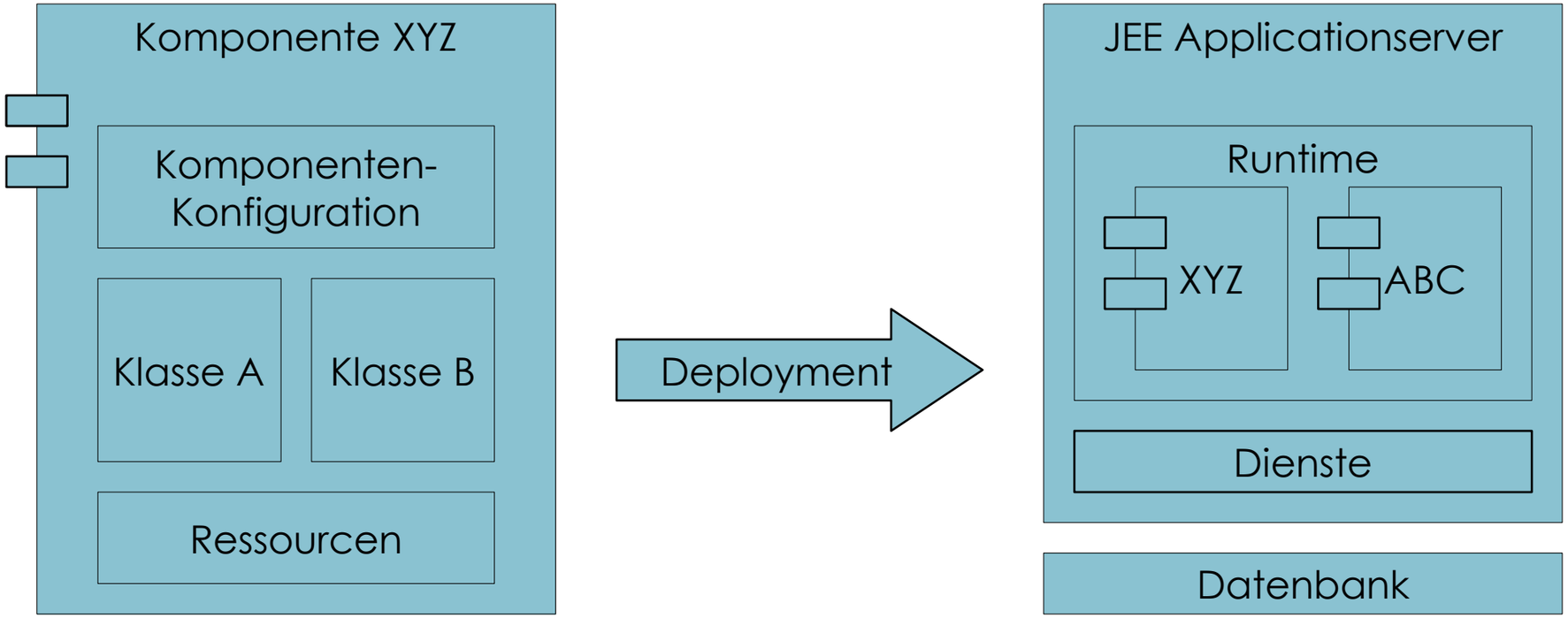
ÜBERSICHT

- Einführung
- JEE Komponenten-Lebenszyklus

DEPLOYMENT VON KOMPONENTEN

- Application Server wird nicht bei jeder Änderung neu gestartet
  - Produktionsbetrieb darf nicht zu stark beeinflusst werden
  - Datenkonsistenz muss immer gewährleistet sein
  - Down Times kosten Geld!
- Integration neuer oder geänderter Komponenten erfolgt zur Laufzeit!
- Komponenten werden dem Application Server bekannt gemacht
- Übernimmt die Darstellung „*Deployment*“ in der Laufzeitumgebung
- Bestimmt, wann Komponente instanziiert wird
- Löst alle Abhängigkeiten zu anderen Komponenten, Diensten und weiteren Ressourcen auf
- Entwickler hat darauf keinen direkten Einfluss

DEPLOYMENT VON KOMPONENTEN



PAKETIERUNG, DEPLOYMENT IN JEE

- Deployment erfordert spezielle Verpackung der Komponenten
- Erfolgt in Java Archiven (*.jar)
  - Fassen viele Dateien in komprimierter Form zusammen
  - Java Klassen, Konfigurationen, externe Bibliotheken, Ressourcen
- Als Module, Anwendungen oder *Assemblies* bezeichnet
- Nach Deployment von der Laufzeitumgebung verwaltet/ausgeführt
- Web-Archive (*.war) enthalten zusätzlich Webressourcen
- Enterprise-Archive (*.ear) enthalten Vielzahl von Komponenten
- Wird i.d.R. durch Tools wie Maven realisiert

LEBENSZYKLUS VON EJB-KOMPONENTEN

- **Entwickler legt keinen Zeitpunkt der Instanziierung fest!**
- Lebenszyklus-Management ist Dienst der Laufzeitumgebung
- Lebenszyklus = vordefinierte Menge von Zustandsübergängen
- Abhängig vom Typ der Komponente
- Zustandsänderungen mittels Callback-Methoden behandelbar

INSTANZIIERUNG DURCH LAUFZEITUMGEBUNG

- Entsprechend Konfiguration
  - Definition als Komponente oder normales Objekt
  - Instanziierung beim Deployment der Komponente
- Als Reaktion auf auslösende Ereignisse, z.B. Web-Anfrage
  - In Abhängigkeit vom Lebenszyklus der Komponente
  - Performanz und Lastverteilung
- In Abhängigkeiten von anderen Komponenten
  - Referenzen auf andere Komponenten aus der Laufzeitumgebung
  - Ggf. Instanziierung abhängiger Komponente
  - Implementierung durch *Dependency Injection* mit *Annotations*

JAVA ANNOTATIONEN

- Sprachelement in Java zur Auszeichnung von Metadaten
  - Auswertung durch Compiler für Warnungen/Fehlermeldungen
  - Auswertung zur Laufzeit mittels *Reflection*
- Starker Einsatz in JEE zur Realisierung des Komponentenframeworks
- Verwendung an Klassen, Methoden, Attributen

```
@Autor("Pascal")
public Class TestBean{
      ...
}
```

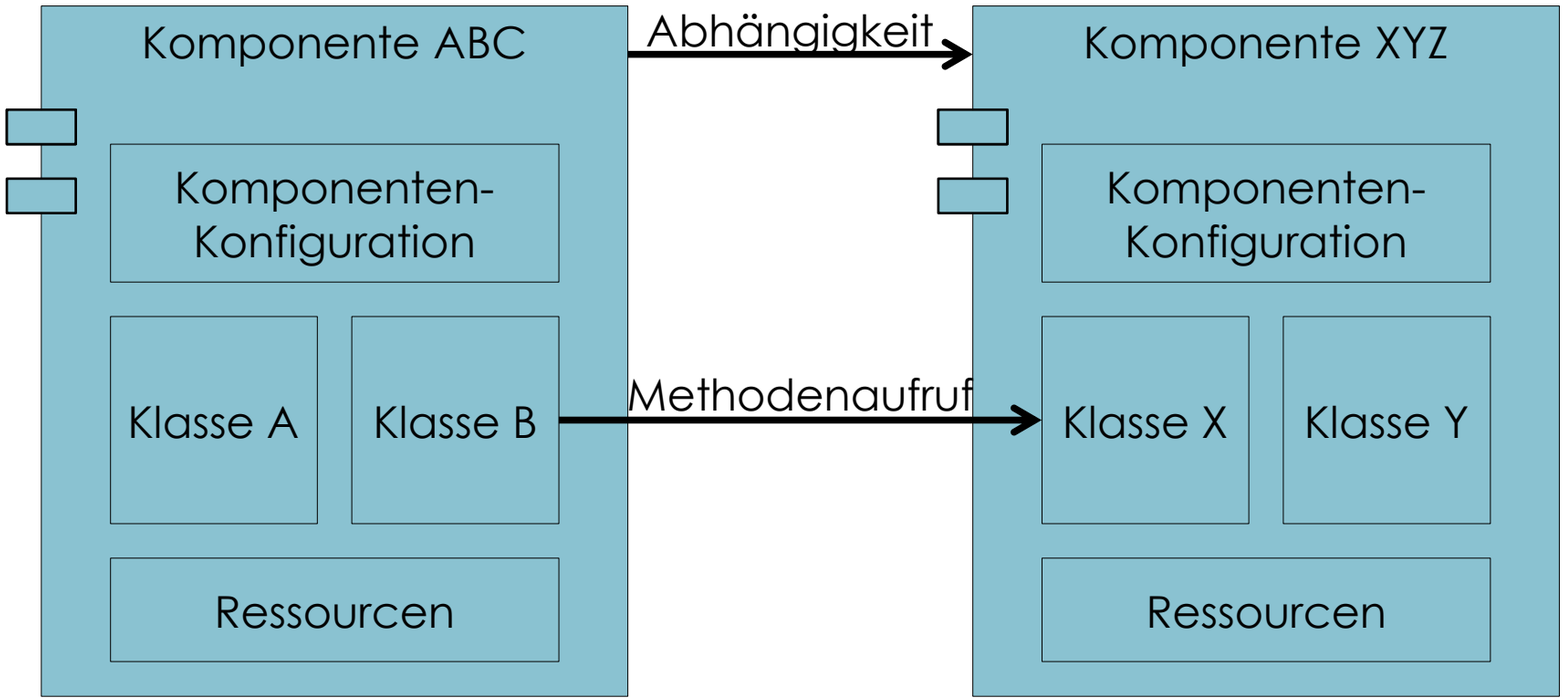
- Definition als spezielles Interface mit parameterlosen und nicht generischen Methoden

```
@interface Autors {
                String value(); // Namen des Autors
}
```

DEPENDENCY INJECTION

- Realisiert Erzeugung und Initialisierung von Objekten und Werten
- Verlagerung der Aufgabe vom Entwickler auf die Laufzeitumgebung
- Dienst zur Minimierung der Abhängigkeiten zwischen Komponenten
  - Vermeidung unnötiger Abhängigkeiten zur Kompilierung
  - Vermeidung fachfremder Implementierung für Tests
- Implementierung und Auflösung in JEE durch Annotationen

DEPENDENCY INJECTION



DEPENDENCY INJECTION

```
public class B{
        private X myX = new X();
}
```

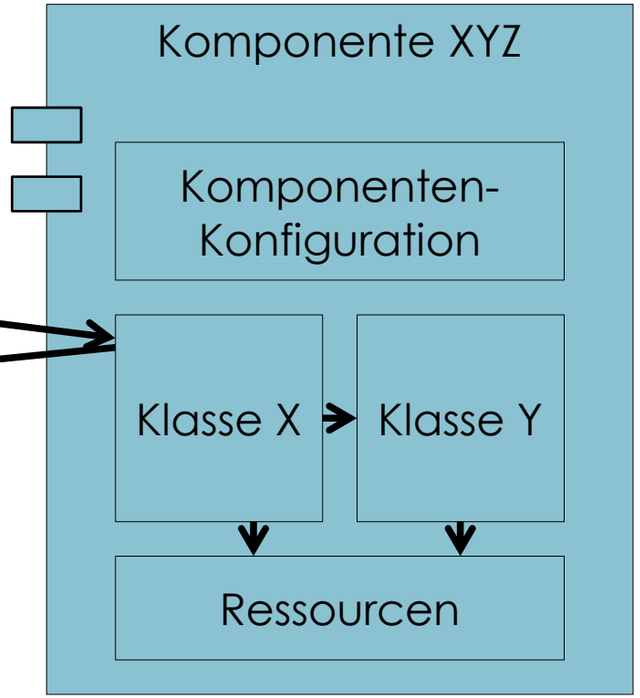
Klasse X

```
public class B{
        private X myX;
        public B(X newX){
                myX = newX;
        }
}
```

```
public class B{
        private X myX;
        static void main(){
                myX = new X();
        }
}
```

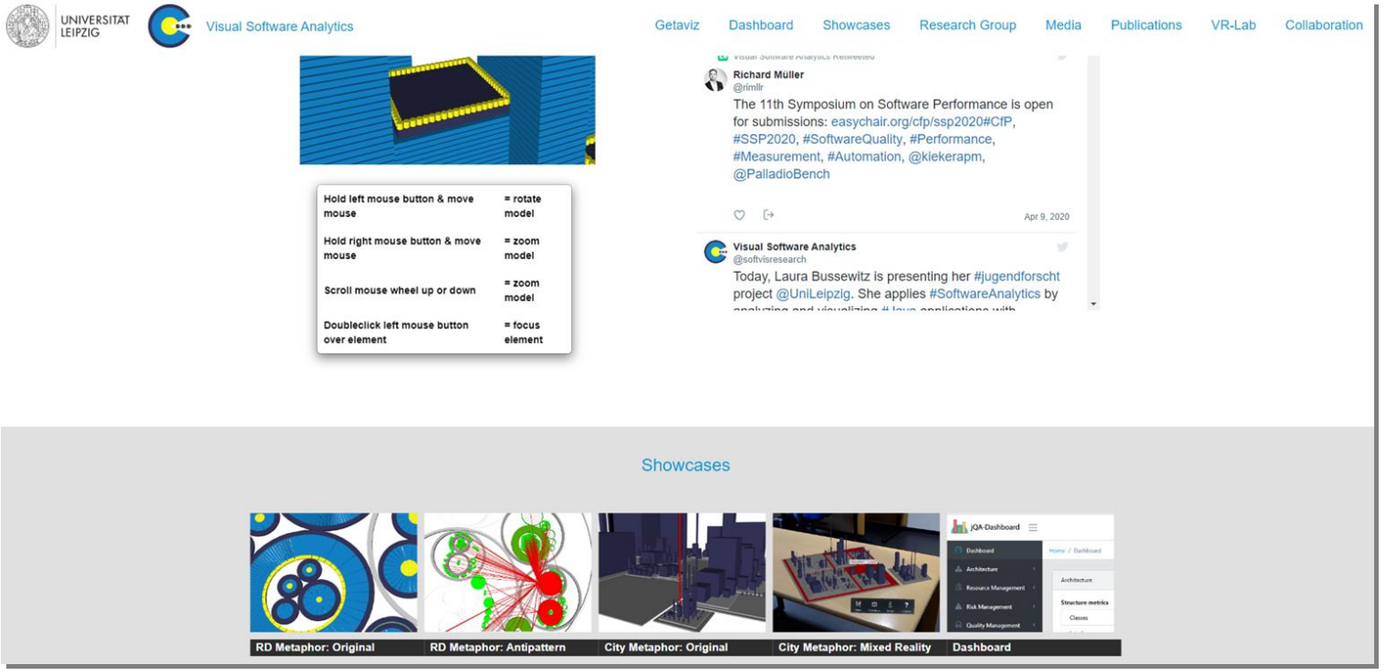
DEPENDENCY INJECTION MIT ANNOTATIONS

```
public class B{

        @EJB
        private X myX;

        public doIT(){
                myX.doAllTheWork();
        }

}
```



UNIVERSITÄT LEIPZIG

VIELEN DANK!



Dr. Pascal Kovacs

Institut für Wirtschaftsinformatik

pkovacs@uni-leipzig.de

https://www.wifa.uni-leipzig.de/iwi/